Libra: A Hybrid-Sparse Attention Accelerator Featuring Multi-Level Workload Balance

Faxian Sun*, Runzhou Zhang, Zhenyu Liu, Heng Liao, Zhinan Qin, Jianli Chen, Jun Yu and Kun Wang†
State Key Lab of Integrated Chips & Systems, and School of Microelectronics, Fudan University, Shanghai, China
*23212020018@m.fudan.edu.cn, †kun.wang@ieee.org

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133063

***Abstract*—Transformers have delivered exceptional performance and are widely used across various natural language processing (NLP) tasks, owing to their powerful attention mechanism. However, the high computational complexity and substantial memory usage pose significant challenges to inference efficiency. Numerous quantization and value-level sparsification methods have been proposed to overcome these challenges. Since higher sparsity leads to greater acceleration efficiency, leveraging both value-level and bit-level sparsity (hybrid sparsity) can effectively exploit the acceleration potential of the attention mechanism. However, increased sparsity exacerbates load imbalance across compute units, potentially limiting the extent of acceleration benefits. To fully exploit the acceleration potential of hybrid sparsity, we propose Libra, an attention accelerator developed through algorithm-hardware co-design. At the algorithm level, we design the bit-group-based algorithm consisting of filtered bit-group sparsification (FBS) and dynamic bit-group quantization (DBQ) to maximize the utilization of sparsity in attention. FBS imposes structured sparsity on weights, while DBQ introduces dynamic sparsification during the computation of activations. At the hardware level, we design task pool to achieve multi-level workload balance, effectively mitigating the load imbalance among compute units induced by hybrid sparsity. Additionally, different stages in DBQ can be executed in parallel, with each stage operating at distinct bit-widths. To support this, we design an adaptive bit-width architecture that enables simultaneous computations at varying bit-widths. Our experiments demonstrate that, compared to state-of-the-art (SOTA) attention accelerators, Libra achieves up to $1.49\times \sim 5.89\times$ speedup and $2.65\times \sim 10.82\times$ enhancement in energy efficiency.**

I. Introduction

In recent years, Transformers [3] have exhibited remarkable performance in Natural Language Processing (NLP) [4, 10, 6, 7, 8, 5, 15], establishing themselves as the prevailing architecture for various tasks. This success is largely attributable to their innovative attention mechanism. The emergence of large pre-trained language models, such as GPT [8, 10], has further entrenched the dominance of Transformer architectures in the field. However, as these models grow in size and complexity, Transformers face increasingly significant computational challenges. The attention mechanism, in particular, imposes substantial computational costs and results in considerable memory footprints, thereby restricting the deployment of Transformer models in resource-constrained devices and real-time applications.

Attention mechanism analyzes the relationships between all tokens in input sequence to evaluate their relative significance through attention scores. However, as the input sequence length grows, the computational complexity and memory cost of attention scores increase quadratically. As a result, attention computation becomes the primary bottleneck in deploying Transformers. To address the high computational redundancy inherent in attention calculation, sparse attention is an appealing approach. Sparse attention reduces the computational cost of dense attention by skipping attention calculation for tokens with low scores, thereby eliminating redundancy in the process. Sanger [14] mitigates some of this redundancy through threshold filtering, setting low-relevance scores to zero. However, it focuses solely on activation computation, neglecting the significant redundancy present in weights. By leveraging sparsity in both weights and activations, the overall redundancy in attention computation can be more effectively reduced.

†Corresponding author

Existing Transformer accelerators [14, 17, 20] leverage value-level sparsity to skip computations involving zero-value correlations, effectively reducing the computational load and achieving acceleration. However, as shown in Fig. 1, even among non-zero values, numerous zero bits remain. This naturally suggests leveraging bit-level sparsity to further reduce inference latency. Computing an N bits data element by only one multiplier unit requires N cycles without accounting for bit-level sparsity. However, with bit-serial architecture (BSA), the number of required cycles matches the number of significant bits, which is always less than or equal to N. However, previous attempts to accelerate using BSA were tailored for Convolutional Neural Networks (CNNs) and are not hardware-friendly for Transformers, making direct adaptation infeasible. This poses significant challenges to our efforts to implement both value-level and bit-level sparsity (hybrid-sparsity) in attention.

In bit-serial computation, workload distribution across compute units is often imbalanced due to the uneven distribution of bit sparsity. As stated in Fig. 1, compute units with lighter workloads may experience extended stalling, waiting for those with heavier workloads to complete. This issue is particularly evident when employing bit-level sparsity in activations. The root of the problem lies in fact that data channels remain unchanged after initial scheduling until all computations are complete, leading to stalled computational resources without further rescheduling. By employing structured sparsity to control distribution of effective bits, bit-level sparsity in weights can mitigate workload imbalance.

To tackle the insufficient utilization of sparsity in attention computation and the workload imbalance caused by hybrid sparsity, we propose Libra, a hybrid-sparse attention accelerator that achieves workload balance through bit-group, offering a solution to make Transformer acceleration more efficient. Our main contributions can be summarized as follows:

- We propose the bit-group-based algorithm consisting of filtered bit-group sparsification (FBS) and dynamic bit-group quantization (DBQ), applying structured sparsity to weights and dynamic sparsity to activations, thereby maximally eliminating redundancy in attention computation.
- An adaptive bit-width hardware architecture is developed to leverage the structural characteristics of the bit-group-based algorithm, allowing different bit-width computation stages to execute in parallel and improving hardware utilization.
- A multi-level workload balance scheme centered around the task pool for workload distribution is introduced. By decomposing workloads into fine-grained tasks and dynamically scheduling them, the scheme achieves near-optimal performance when exploiting hybrid-sparsity.
- Extensive experiments are conducted to evaluate the performance of our accelerator. Libra achieves up to $1.49\times \sim 5.89\times$ speedup and $2.65\times \sim 10.82\times$ enhancement energy efficiency compared to SOTA attention accelerators with negligible accuracy loss.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.

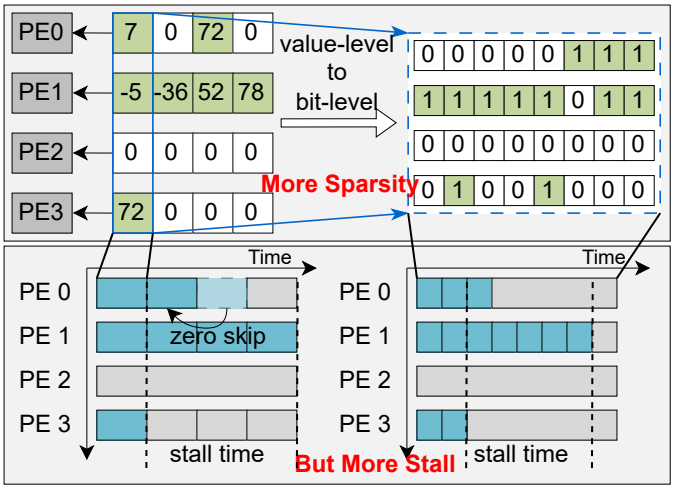


Fig. 1: Increased workload imbalance issue when leveraging bit-level sparsity.

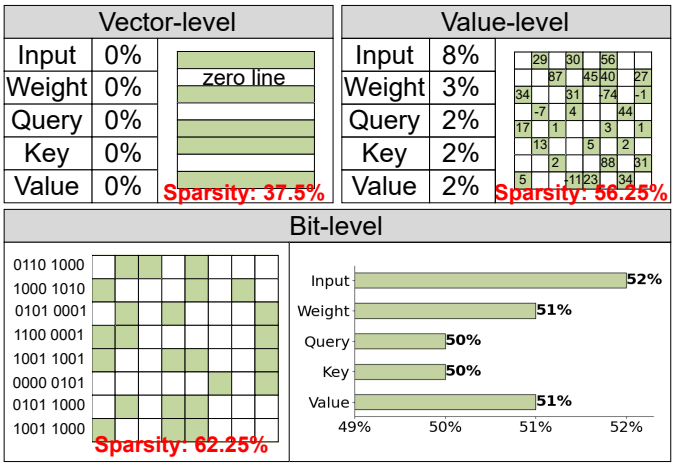


Fig. 2: Preliminary experimental results on the vector-level, value-level and bit-level sparsity in the BERT model and GLUE dataset.

II. BACKGROUND AND MOTIVATION

A. Attention Mechanism

The attention mechanism is the core operation within the Transformer models. During the computation of attention, the embedded input sequence is initially processed through linear layer to generate the Query (Q), Key (K), and Value (V) matrices. After that, the score matrix is derived by computing the dot product between Q and the K^T. Subsequently, the generated score matrix is normalized using non-linear functions to produce the probability matrix P which is then multiplied with the V. The result matrix is applied to linear functions and derived through residual connection. After the final layer normalization, we can get the attention result.

B. Sparse Accelerator

Various sparsity and quantization methods have been employed to accelerate attention computations. Sanger [14] leverages redundancy in the attention mechanism to enhance value-level sparsity. DTQAtten [17] and DTATrans [16] implement token-wise mixed-precision quantization. DEQ [20] further advances this by introducing element-wise mixed-precision quantization. These methods have achieved remarkable performance. However, they only optimize at the value level, neglecting the huge potential for acceleration at the bit level.

Existing researches on bit sparsity focus on accelerating CNNs using BSA. Laconic [9] processes both weights and activations as bit sequences, while Adas [18] dynamically selects the side with fewer effective Booth terms for sequence expansion. However, the irregular distribution of bit sparsity in weights and activation can result in load imbalance during computation, where some processing elements (PEs) are active while others remain idle. While Adas uses inner-join units to alleviate this issue, it does not fully resolve the problem for the entire PE array. To address the issue of load imbalance, Bit-Balance [19] proposes a channel-wise bit-sparse quantization method and uses clustering techniques to balance workloads across PEs.

Despite these advances, the aforementioned bit-sparsity methods primarily rely on patterned distributions of significant bits in weights, which are less applicable to the dynamically generated activations in Transformers. BSViT [21] introduces a dense-sparse hybrid dataflow method to achieve workload balance in PEs but primarily leverages dynamic patch quantization in Vision Transformers (ViT) [11] and lacks a comprehensive focus on the unique characteristics and requirements of the attention mechanism in Transformers.

C. Motivation

The primary motivation for our work is based on two key observations. On the one hand, as shown in the Fig. 2, Transformers exhibit extremely high bit-level sparsity, approximately 25× greater than value-level sparsity without applying quantization or sparsity techniques. In particular, the bit-level sparsity in P can reach as high as 96%. Therefore, leveraging hybrid sparsity will further enhance the acceleration effects by adding bit-level sparsity on top of value sparsity. Additionally, due to the redundant nature of the attention mechanism, combining hybrid sparsity with attention features holds significant potential for further acceleration.

On the other hand, as illustrated in Fig. 1, the irregular distribution patterns of zero values and zero bits increase the likelihood of compute units idling, which further exacerbates workload imbalance. The acceleration benefits brought by increased sparsity may not always offset the losses caused by load imbalance. Therefore, maintaining balanced workloads while leveraging hybrid sparsity remains an unavoidable challenge. For instance, Bit-Balance [19] uses a clustering method to group workloads with similar characteristics into the same channels, facilitating inter-channel load balance through scheduling. Adas [18] achieves load balance within PEs through inner-join units. However, when leveraging hybrid sparsity, single-level load balance methods are insufficient to address these challenges. This motivates the design of our multi-level workload balance strategy.

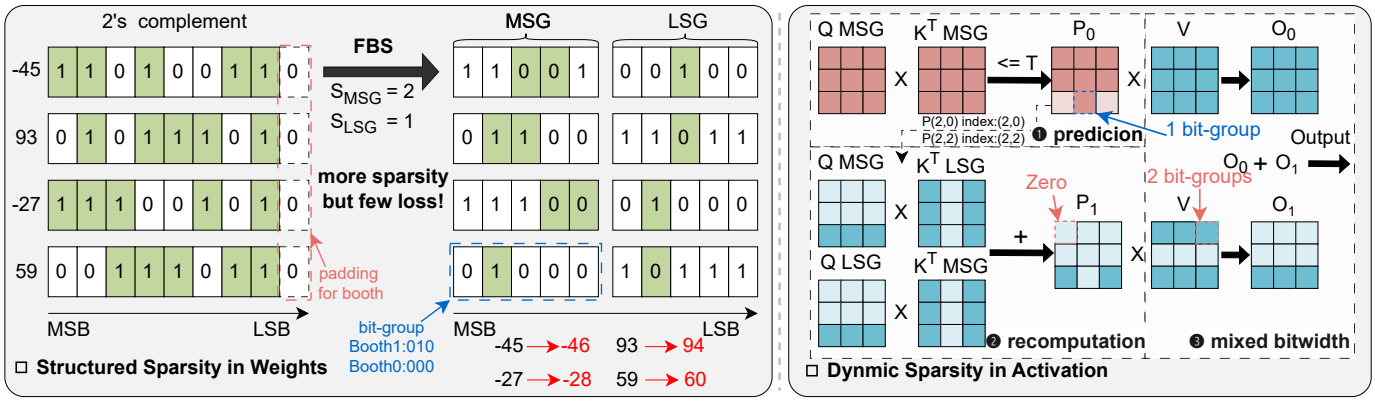
III. BIT-GROUP-BASED ALGORITHM

In this section, we introduce our algorithmic optimizations based on bit-group, extending from weight FBS to activation DBQ. BSA performs a significant amount of unnecessary computations due to the presence of negative numbers [9]. To eliminate these redundancies, we incorporate Booth encoding into our bit-level operations. Furthermore, to fully leverage the advantages of Booth encoding, our algorithm is structured around bit-group, each containing 5 bits that can be encoded into two Booth codes.

A. Filtered Bit-group Sparsification

We propose the FBS scheme to reduce redundancy in weight representations and decrease hardware computational overhead. For integers encoded with Booth encoding, only bit sequences of '000' and '111' are encoded as '0'. Therefore, increasing the continuity of bit positions in the weights can enhance the sparsity of weights after Booth encoding.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.



(a) Filtered Bit-group Sparsification (b) Dynamic Bit-group Quantization

Fig. 3: An overview of bit-group-based algorithm. (a) Libra takes filtered bit-group sparsification (FBS) to balance weight distribution with negligibe accuracy loss. (b) Libra employs dynamic bit-group quantization (DBQ) to introduce dynamic sparsity in activations.

Algorithm 1: Filtered Bit-Group Sparsification

Input: Weight tensor $\mathbf{W}$, error threshold ϵ
Output: Compensated sparsified weight tensor $\mathbf{W}_{\text{comp}}$
$\mathbf{W}_q \leftarrow \text{Quantize}(\mathbf{W},\ 8\ \text{bits})$ ▷ Step 1
  Extract high bits (**MSG**) and low bits (**LSG**):
  MSG ← high 5 bits of $\mathbf{W}_q$;
  LSG ← low 4 bits of $\mathbf{W}_q$;
  LSG ← left-shift **LSG** by 1 bit;
Sparsify **MSG** and **LSG** under constraints: ▷ Step 2
  $\text{MSG}_{\text{sparse}} \leftarrow \text{Sparsify}(\textbf{MSG},\ S = 2)$;
  $\text{LSG}_{\text{sparse}} \leftarrow \text{Sparsify}(\textbf{LSG},\ S = 1)$;
Reconstruct sparsified weights: ▷ Step 3
  $\mathbf{W}_{\text{sparse}} \leftarrow \text{Combine}(\text{MSG}_{\text{sparse}},\ \text{LSG}_{\text{sparse}})$;
Compute error: $\Delta\mathbf{W} \leftarrow \mathbf{W}_q - \mathbf{W}_{\text{sparse}}$ ▷ Step 4
while $\|\Delta\mathbf{W}\|_2 > \epsilon$ **do**
  Adjust $\text{MSG}_{\text{sparse}}$, $\text{LSG}_{\text{sparse}}$ to minimize $\|\Delta\mathbf{W}\|_2$ under constraints;
  Reconstruct $\mathbf{W}_{\text{sparse}}$;
  $\Delta\mathbf{W} \leftarrow \mathbf{W}_q - \mathbf{W}_{\text{sparse}}$;
$\mathbf{W}_{\text{comp}} \leftarrow \mathbf{W}_{\text{sparse}} + \text{CompensateError}(\Delta\mathbf{W})$;

To better preserve the numerical value of the weights, each quantized weight can be represented as $B = -B_{n-1}2^{n-1} + \sum_{i=0}^{n-2} B_i * 2^i$. Numerical values are composed of the sum of multiple powers of two. Therefore, the weights can be denoted as follows,

$$B = \begin{cases} \sum_{i=m}^{M} B_i * 2^i, & B \geq 0 \\ -\sum_{i=m}^{M} B_i * 2^i - 1, & B < 0 \end{cases}, \quad (1)$$

where $M = \min(N-1, m+s-1)$, s represents the length of the continuous range of effective bits, and $m \in \{0, 1, 2, \ldots, N-1\}$. As shown in Fig. 3 and Algorithm 1, the quantized weights are first divided into two groups. The first group, called Most Significant Group (MSG), includes bits from the 7th to the 3rd position. The second group, called Least Significant Group (LSG), includes bits from the 3rd to the 0th position, with an additional zero appended at the end according to Booth encoding rules.

Next, we assign a fixed maximum number of significant bits, S, to each bit group. For example, $S = 2$ for MSG and $S = 1$ for LSG. For positive integers, the significant bits are the non-zero bits, but for negative integers, the significant bits are the zero bits. Finally, we optimize the weights of the current layer according to the constraints defined by our scheme. If the difference between the sparsified weights and the original weights exceeds a given threshold ϵ, we introduce appropriate compensations and re-sparsify them iteratively until the specified constraints are satisfied.

B. Dynamic Bit-group Quantization

In Transformers, the correlation between each pair of tokens is initially calculated as $P = Q \cdot K^T$. Due to the softmax function, less significant correlations P approach zero, making them irrelevant in the subsequent $P \cdot V$ computations. Therefore, we can predict the value of P using lower-cost computations, assign lower bit-width precision to less important values, and perform high-precision recalculations for important values. This approach minimizes computational complexity while maintaining accuracy.

In our design, we introduce a Dynamic Bit-group Quantization (DBQ) scheme, which can dynamically allocate data bit-widths without requiring a redesign of the network architecture. Since i-BERT [13] can maintain accuracy comparable to full precision using Int8, our design also adopts Int8 as the base precision. As shown in Fig. 3, the DBS scheme consists of two main stages.

The first stage is the prediction stage. To be compatible with the DBQ scheme, the MSG of Q and K are multiplied to generate a prediction result P, which is represented by a single bit-group. Each element of P is then compared to a predefined threshold T. The threshold T is determined by a ratio parameter r and the specific task. If $P_{i,j} \leq T$, the corresponding correlation is considered unimportant and can be represented with a single bit-group. Otherwise, a high-precision recalculation is required to avoid accuracy loss. To facilitate locating important elements, we retain the row and column indices for future use.

The second stage is the recomputation stage. Using the previously stored indices, we can locate the important elements $P_{i,j}$, retrieve the ith token of Q and the jth token of K, read their MSG and LSG, and then perform vector-vector multiplication to obtain $P_{i,j}$. This recalculation process is repeated for all important elements. Now, we have a matrix P with two different precision, which needs to be multiplied by the value matrix V. Considering hardware implementation specifics, we split this into two groups of computations: one for the full-precision p multiplied by V, producing a partial output matrix O_0. The other group, based on the previously stored indices, retrieves the high-precision $P_{i,j}$ and the jth token of V, multiplying them to produce another part of the output matrix O_1. The final output O is obtained by summing the two matrices.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.

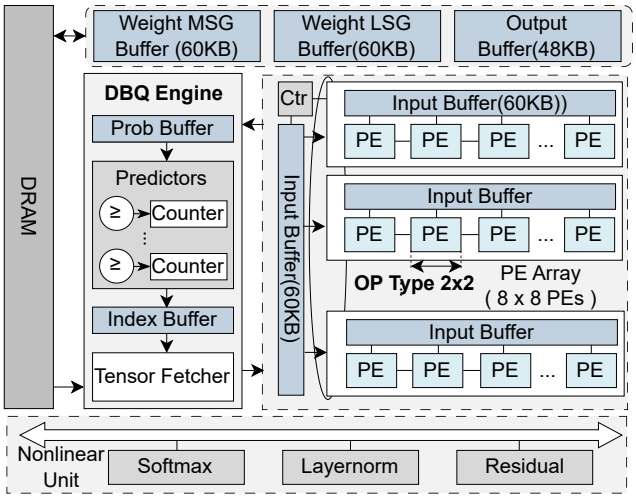


Fig. 4: Overview of Libra hardware architecture.

IV. LIBRA HARDWARE ARCHITECTURE

A. Architecture Overview

To fully leverage the advantages of FBS and DBQ, we designed Libra, a hybrid-sparse attention accelerator closely tailored to the algorithm. The key architectural highlights include: 1) an adaptive bit-width PE based on the bit-group multiplier unit (BMU), which accommodates the varying bit-width requirements at different computation stages. 2) a task pool, which maximizes bit sparsity utilization, effectively balances workloads across multi-levels. In this section, we first present the overall architecture of Libra. Then we discuss how the task pool within the BMU reduces unnecessary computations by exploiting bit sparsity. Finally, we detail the PE constructed from BMU, highlighting its ability to leverage hybrid sparsity and implement multi-level workload balance strategies. The overall architecture of Libra, as shown in Fig. 4, includes input buffers, weight buffers, an output buffer, an 8×8 decoupled PE array composed of adaptive bit-width PEs, a global accumulator, nonlinear units, a DBQ engine, and a top-level controller.

B. Bit-group Multiplier Unit

As shown in Fig. 6, a single PE unit consists of four BMU. Each BMU is composed of an encoder, a task pool, four bit-group compute cores, and decoder. The input to each BMU is bit-groups. As shown in Fig .5, after being encoded by the encoder, tasks are generated and stored in the task pool for computation, with the output handled by the accumulation unit. In the task generation phase, our hardware unit eliminates unnecessary 0-Booth computations, taking advantage of bit sparsity.

1) **Encoder**. We ensure that each BMU receives data represented by a single bit-group, which allows us to implement our encoder with very low overhead. The encoder receives bit-group in each cycle, producing necessary task pairs that are sent to the task pool. The bit-group can be encoded into two Booth-encoded values according to the following formulas: $Booth_1 = 4 * (-2b_4 + b_3 + b_2)$, $Booth_0 = 1 * (-2b_2 + b_1 + b_0)$. The values of b_i range from {2, 1, 0, -1, -2}, so $Booth_i$ has a range of $[-8, 8]$, and can be shown as $[-2^3, 2^3]$. So we can decode these into a sign bit, a valid bit and a 3-bit shift term. When performing multiplication, only the exponent terms need to be added. Finally, the decoder produces the final output.

During the task generation stage, we use valid bit to compress unnecessary computations. For example, as shown in Fig. 5, for A and B, with valid bits of [1, 0] and [1, 1], respectively, if the Booth_0

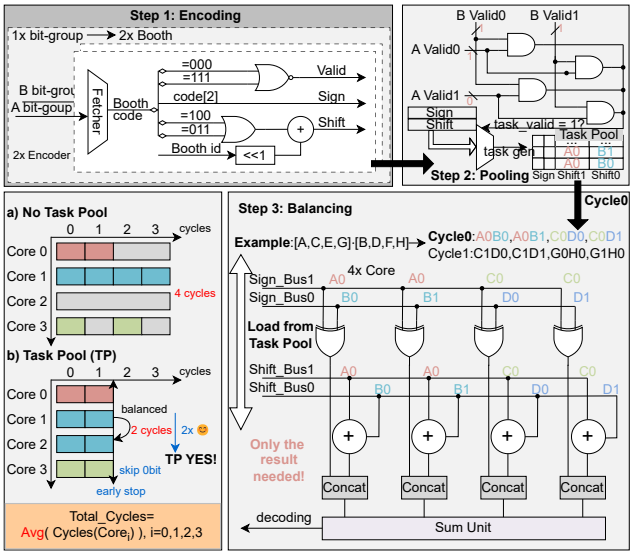


Fig. 5: Illustration of workload balancing in BMU via task pool.

of A is 1, two tasks $[A_0, B_1]$ and $[A_0, B_0]$ will be generated. This reduces the required tasks by 50% compared to the original four tasks.

2) **Task Pool**. The task pool is primarily used to temporarily store tasks, enabling fine-grained scheduling both within and between BMU to fully utilize sparsity. Since each BMU contains four compute cores, the task pool must maintain enough tasks to ensure all cores remaining active, to prevent idle periods. Simultaneously, since up to 4 tasks can be generated in a single decode cycle, we designed the task pool of each BMU to have a depth of 8, striking a balance between functionality and minimizing overhead.

For each BMU, the computation involves multiplying and summing two vectors. As we only need the correct final result rather than element-wise result, we can use the task pool to issue tasks out-of-order, dynamically assigning them to compute cores that have finished their previous tasks. This avoids load imbalance caused by the irregular distribution of bit sparsity within the BMU.

3) **Compute Core**. The input to the compute core is a task. Due to the decoder design, we can use an adder to achieve the same functionality as a multiplier. We perform an XOR on the sign bits to obtain the sign of the partial product, while the exponent terms are added to determine the final shift amount. Since we only need the sum of all partial products, there is no need for immediate decoding. The sum unit accumulates the results from all cores based on the sign bits, and once all computations are complete, the decoder provides the final output.

C. BMU-based Adaptive PE

In the computation process, we encounter three different bit-width operations: 2×2, 2×1, and 1×1, where '1' represents a single bit-group (3 bits) and '2' represents 2 bit-groups (5 bits). To avoid wasting area by designing separate PE units for each bit-width, we designed an adaptive bit-width PE unit based on BMU. By combining BMUs in different configurations, we support three different bit-width operations required by our algorithm.

As shown in the Fig. 6, we can combine 2 BMU into one 2×1 PE. Similarly, 4 BMU can be further combined into a 2×2 PE. Taking 2×2 computation as an example, there is some sparsity at the value level. Thus, before feeding data into the PE, we perform bit-mask preprocessing to eliminate unnecessary computations at the value level. Bit-level sparsity utilization is handled by the task pool,

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.

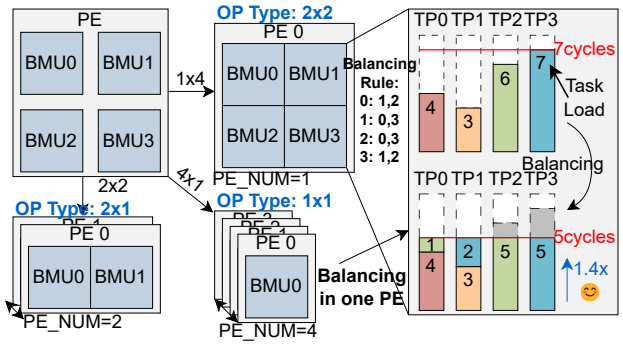


Fig. 6: Cases of adaptive bit-width in PE. Workload balancing within PE for 2x2 OP type.

as described in the previous section. Due to the irregularity of the hybrid sparsity distribution, we found that maintaining workload balance among BMUs within a PE and across different PEs is difficult through pre-allocation alone. Therefore, we implement a multi-level workload balance strategy using the task pool from the BMUs. Each PE contains 4 task pools, which are arranged in a loop to enable data sharing between 4 BMUs. To further reduce the overhead caused by task scheduling, tasks are prioritized within the same PE. As shown in Fig. 6, when BMU0 completes its computation, it loads task from the task pool of the adjacent BMU at the tail end, performs the calculation, and temporarily stores the accumulated result in a dynamic register. Once all computations are completed, the result is returned to the corresponding BMU for summation, allowing for any subsequent operations such as shifting.

Similarly, load imbalance can occur across PEs. We set up a stack in each PE's input channel to temporarily store input data loaded onto the chip. A monitor is added to stack to track the number of value pairs expected to be computed by each PE in real time. Within the same PE group, stacks are shared. When one PE finishes its computation, it will prioritize reading from the stack of the PE with the most remaining tasks, thereby balancing the workload within the same PE group. Considering that in the $P \cdot V$ computation stage, different PE groups handle different types of computations, and cross-PE group scheduling would incur significant overhead, outweighing the potential benefits. Therefore, we do not consider scheduling across PE groups.

V. EVALUATION

A. Experimental Setup

Benchmarks. To validate the computing performance of Libra, we select the Bert-base model and fine-tuned it on eight GLUE text classification tasks. Our evaluation criteria encompass F1 scores for MRPC and QQP, Pearson correlation for STS-B, and Matthews correlation for CoLA. Accuracy on the validation set is reported for remaining tasks. Subsequently, we evaluate the model's inference capabilities, building them with the open-source Huggingface [12] and PyTorch [2] libraries. For all models, we perform five rounds of fine-tuning using the AdamW optimizer with a learning rate of 2e-6 and cosine scheduling.

Hardware implementation. We implement the Libra architecture in Verilog HDL and synthesize it by Synopsys Design Compiler at 500MHz under TSMC 28nm HPC+ technology. The power consumption of SRAM-based on-chip systems is obtained by CACTI 7 [1].

B. Evaluation on Libra Algorithm

1) Weight FBS. To evaluate the effectiveness of the proposed FBS for weights, we test the Booth sparsity of weights after applying FBS and compare the results with a baseline without FBS. Fig. 7 shows the Booth sparsity across different datasets. We apply '2' and '1' constraints to MSG and LSG, respectively. Compared to the baseline, Booth sparsity in weights after FBS increase by an average of 15.92%. Given that, without FBS, the computational reduction in Bit-Serial Architecture depends on bit sparsity, we compare the Booth sparsity of weights after FBS with the bit sparsity of baseline weights, observing an average increase of 13.41%. While significantly enhancing sparsity, the accuracy loss due to the FBS algorithm remains negligible. This is because FBS leverages the characteristics of Booth encoding to filter out a large number of redundant consecutive '1's in negative values. Additionally, the introduction of bit-group reduces the impact on larger values and allows for more aggressive sparsification of less influential values.

TABLE I Accuracy Results of Different Approaches on Different Tasks

TASK	QNLI	MRPC	MNLI	QQP
Baseline	91.1	89.8	84.7	87.9
Sanger	91.0	88.7	84.8	87.9
DTQAtten	90.5	89.7	85.0	75.5
DEQ	90.8	89.6	85.2	87.7
Libra	**91.1**	**88.6**	**83.6**	**88.1**
TASK	**SST-2**	**CoLA**	**RTE**	**STS-B**
Baseline	92.8	55.7	63.2	88.5
Sanger	93.1	57.2	66.4	88.6
DTQAtten	92.2	54.3	63.0	-
DEQ	92.5	58.3	62.4	88.4
Libra	**91.9**	**57.8**	**65.7**	**88.7**

TABLE II Area and Power of Different Modules

Module	Area[mm^2]	Power[mW]
PE Array	0.247	62.53
DBQ Engine	0.004	0.102
Nonlinear - Softmax	0.13	5.63
Nonlinear - Others	0.072	1.085
SRAM Buffer	0.478	268.9
Others	0.199	38.41
Total	**1.13**	**376.66**

2) Activation DBQ. To investigate the effective utilization of activation sparsity potential by DBQ, we evaluate the impact of the ratio parameter r on accuracy loss and the associated reduction in computation across multiple datasets. Fig. 8 illustrates the percentage of computations for $Q \cdot K^T$ that can directly leverage the predicted results under different values of r. And Table III illustrates the accuracy loss. Notably, for QQP at r = 0.8, the proportion of computations that directly used predicted results reached 93.3%, with no accuracy loss. This observation underscores that within attention computation, a substantial portion of calculations are redundant, indicating that only a small subset of token pairs exhibiting high correlation significantly affect accuracy.

Sanger and DTQAtten directly eliminate computations related to token pairs with low correlation, resulting in wasted computational overhead for prediction. We argue that for token pairs with low correlation, the information provided by the prediction is sufficient to replace the result from full-precision computation without causing significant accuracy loss. When the dynamic sparsity ratio is sufficiently high, even the eliminated small values still contain a substantial amount of information. Furthermore, the computational overhead of predicting correlation increases with the input length. Therefore, DBQ reuses the predicted results, allowing fewer high-precision calculations while maintaining the same accuracy loss as Sanger and DTQAtten.

3) Put it Together. Using the optimal configuration, we integrate FBS and DBQ, presenting the final accuracy and corresponding

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.

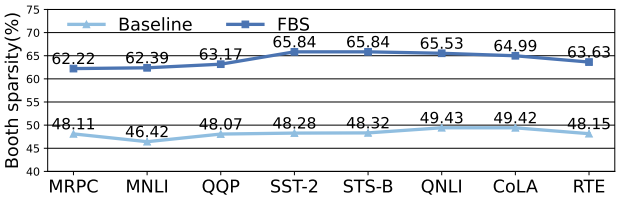


Fig. 7: Average Booth sparsity of linear layer weights across different tasks on Glue.

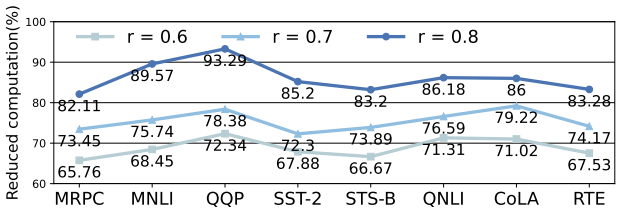


Fig. 8: Reduced computation of different r settings of DBQ.

TABLE III Accuracy Loss of Different r Settings of DBQ.

r	MRPC	MNLI	QQP	SST-2	STS-B	QNLI	CoLA	RTE
0.6	1.01%	0.89%	-6.93%	-0.71%	-3.39%	1.48%	-0.35%	-0.23%
0.7	1.14%	1.12%	-4.31%	-0.53%	-3.35%	1.4%	-0.24%	-0.14%
0.8	1.35%	1.01%	-3.75%	-0.17%	-3.96%	1.25%	-0.17%	-0.04%

speed-up for each dataset in the Table I. The average accuracy loss for the Libra sparse scheme remains under 1%.

C. Evaluation on Libra Hardware

Libra is implemented in TSMC 28nm technology, operating at 500MHz. The detailed area and power breakdown of Libra are shown in Table II. Our accelerator features an 8×8 processing array, with 408 KB of SRAM allocated for weights, inputs, temporary buffers, and their corresponding buffers. Our accelerator have a total area of $1.13mm^2$ and total power of $376.66mW$.

We contact extensive experiments on glue datasets to compare our accelerator with SOTA Transformer accelerators in terms of speedup and energy efficiency. A cycle-accurate simulator is developed to collect performance statistics. Fig. 10 presents the normalized performance with other accelerators. Experimental results show that comparing with Sanger, DTQAtten, DEQ, Libra has achieved $5.89\times$, $3.35\times$, $1.49\times$ normalized performance gain, respectively. As shown in Fig. 11, our accelerator achieves $10.82\times$, $7.41\times$, and $2.65\times$ enhancement in energy efficiency compared with Sanger, DTQAtten, and DEQ, respectively. The improvements are driven by three key factors. First, our algorithm exploits hybrid sparsity, reducing redundant computations caused by zero values and bits. And then, Libra significantly decreases the computational load in the $Q \cdot K^T$ operation, a critical step in attention mechanisms, resulting in enhanced overall efficiency. Lastly, the multi-level workload balance strategy improves the utilization of computational resources and reduces energy consumption.

Fig. 9 illustrates the performance under different task pool depths (0, 4, 8, and 16) across various tasks. We observe that as the depth increases, performance also improves. This trend can be attributed to the enhanced capacity of the task pool to manage workloads, thereby reducing the likelihood of load imbalance. However, increasing the depth also results in greater area and power consumption, as well as increased complexity in scheduling logic. Considering the trade-offs between performance and additional overhead, we set the task pool depth to 8 for optimal balance.

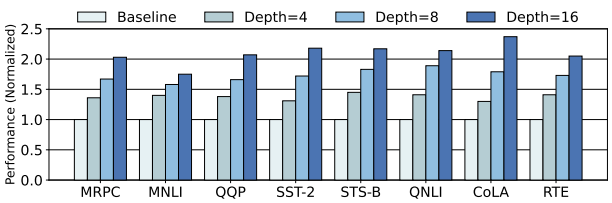


Fig. 9: Performance analysis with different task pool depths. All results are normalized to Baseline (depth=0).

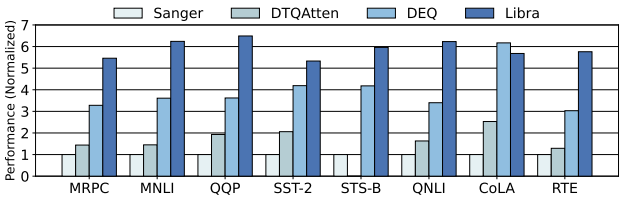


Fig. 10: Comparison of performance between Libra and other accelerators. All results are normalized to Sanger.

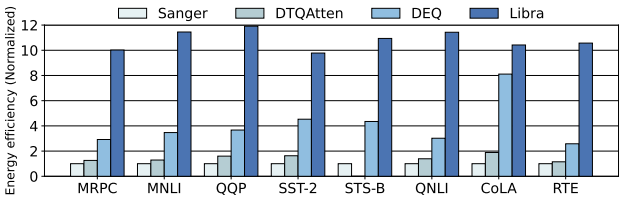


Fig. 11: Comparison of energy efficiency between Libra and other accelerators. All results are normalized to Sanger.

To validate the effectiveness of our multi-level workload balance strategy, we evaluated Libra's performance under same conditions without enabling the multi-level load balance strategy across various tasks. When the task pool depth is 0, the multi-level workload balancing strategy is not enabled. As shown in the Fig. 9, without multi-level workload balance, Libra's performance decreased by an average of $1.73\times$, indicating the strategy's effectiveness. The multi-level workload balance strategy allows Libra to achieve local workload balance within BMUs and between PEs, thereby reducing the stalling of compute units.

VI. CONCLUSION

This paper presents Libra, a software-hardware co-design that leverages hybrid sparsity to accelerate attention-based models. The bit-group-based algorithm applies sparsification to weights, and dynamically adjusting the number of bit-groups for different elements by exploiting the redundancy in the attention mechanism, resulting in a significant reduction in computation. To fully harness the potential of the hybrid sparsity and algorithm, we design a matching adaptive bit-width architecture, with multi-level optimizations to address load imbalance caused by hybrid sparsity. Experimental results demonstrate that Libra can achieve up to $1.49\times \sim 5.89\times$ speedup and $2.65\times \sim 10.82\times$ enhancement in energy efficiency.

ACKNOWLEDGEMENTS

This work was financially supported in part by National Key Research and Development Program of China under Grant 2021YFA1003602, in part by Project of Shanghai EC under Grant 24KXZNA12, and in part by National Natural Science Foundation of China (NSFC) Research Project 92373207. The computations in this research were performed using the CFFF platform of Fudan University.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Rajeev Balasubramonian et al. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Transactions on Architecture and Code Optimization (TACO)* 14.2 (2017).

[2] Adam Paszke et al. Automatic differentiation in pytorch (2017).

[3] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[4] Jacob Devlin. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[5] Alec Radford. Improving language understanding by generative pre-training (2018).

[6] Z Lan. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942* (2019).

[7] Yinhan Liu. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[8] Alec Radford et al. Language models are unsupervised multitask learners. *OpenAI blog* 1.8 (2019).

[9] Sayeh Sharify et al. Laconic deep learning inference acceleration. In *Proceedings of the 46th International Symposium on Computer Architecture*. 2019.

[10] Tom B Brown. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).

[11] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

[12] Thomas Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 2020.

[13] Sehoon Kim et al. I-bert: Integer-only bert quantization. In *International conference on machine learning*. PMLR. 2021.

[14] Liqiang Lu et al. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 2021.

[15] Shaden Smith et al. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990* (2022).

[16] Tao Yang et al. DTATrans: Leveraging dynamic token-based quantization with accuracy compensation mechanism for efficient transformer architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42.2 (2022).

[17] Tao Yang et al. Dtqatten: Leveraging dynamic token-based quantization for efficient attention architecture. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE. 2022.

[18] Xiaolong Lin et al. AdaS: A Fast and Energy-Efficient CNN Accelerator Exploiting Bit-Sparsity. en. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. San Francisco, CA, USA, 2023.

[19] Wenhao Sun et al. Bit-balance: Model-hardware co-design for accelerating nns by exploiting bit-level sparsity. *IEEE Transactions on Computers* (2023).

[20] Xuhang Wang et al. Deq: Dynamic element-wise quantization for efficient attention architecture. In *2023 IEEE 41st International Conference on Computer Design (ICCD)*. IEEE. 2023.

[21] Gang Wang et al. BSViT: A Bit-Serial Vision Transformer Accelerator Exploiting Dynamic Patch and Weight Bit-Group Quantization. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2024).

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:23 UTC from IEEE Xplore. Restrictions apply.